시스템 분석 및 설계

3주차. 요구 분석

서울사이버대학교

학습목표

1. 요구 분석은 어떤 작업이며, 그 과정을 이해하고 설명할 수 있다.
2. 요구가 무엇이며 기능요구와 비 기능 요구를 구별하고 찾아낼 수 있다.
3. 문제 분석 단계에서 필요한 다양한 실무적 관점을 이해한다.

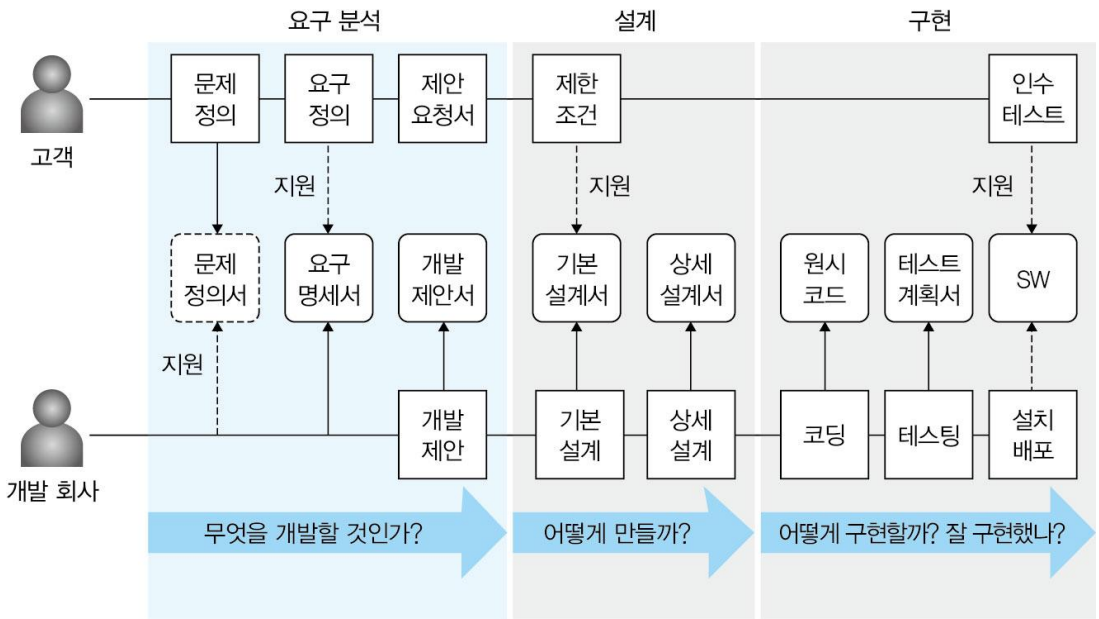
폭포수 개발 모델 뿐 아니라 대부분의 개발 업무의 첫 단계는 사용자 요구를 분석하고 정의하는 것으로, 개발 초기에는 모호할 수 있는 요구를 요구사항으로 구체화하는 요구분석 단계는 시스템 개발에서 가장 어렵고 중요한 과정

- 요구 분석은 구축하거나 수정할 소프트웨어에 대해 사용자의 기대사항을 정의하는 프로세스
- 다양한 사용자의 요구를 찾아내고 인식하기 위해 수행되는 모든 작업을 통칭
- 따라서 요구 분석에는 사용자를 비롯하여 관련자의 참여가 반드시 필요하며, 개발자* 는 소프트웨어 요구사항을 분석, 문서화, 검증해야 함

요구분석-설계-구현으로 이어지는

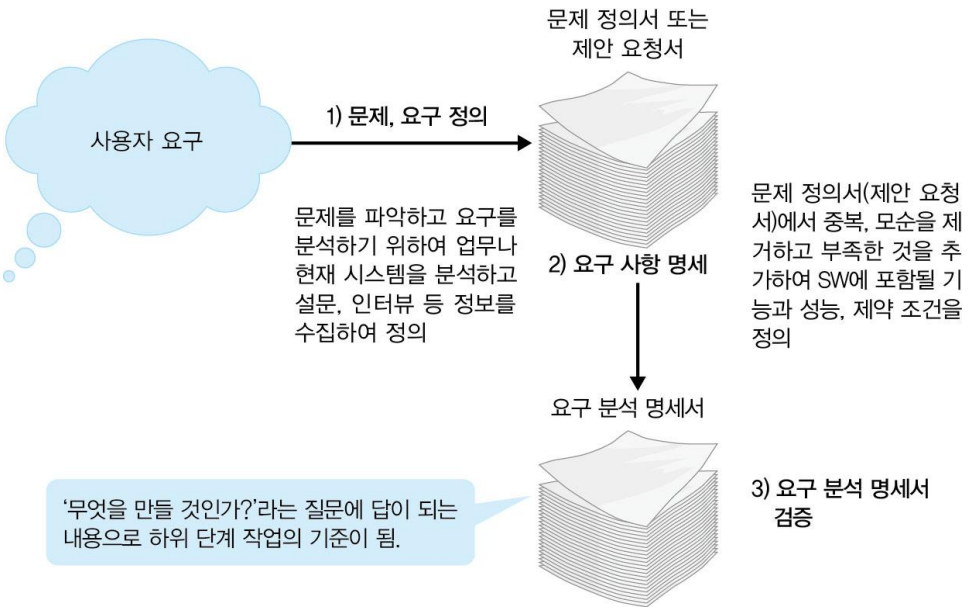
폭포수 모델을 따라 소프트웨어를 개발하는 경우

그림과 같은 과정을 거쳐 사용자와 협력



- 소프트웨어를 발주한 고객은 프로젝트가 시작되는 요구 분석 단계에서 현황에 대한 파악과 문제점을 분석하는데 도움을 주어야 함, 즉 문제 분석과 요구 파악을 위해 개발자에게 여러가지 정보를 제공함
- 쓸모 있고 사용이 쉬운 소프트웨어를 개발하기 위해 사용자가 요구 분석에 참여하는 것은 필수적
- 이는 프로젝트의 성공과 만족도에 크게 영향을 줌
- 또한 프로젝트 초기에 사용자가 참여하여 개발자와 함께 문제를 분석하고 요구를 파악하는 것은 소프트웨어 품질에 크게 영향을 주기 때문에 사용자의 역할이 매우 중요

- 소프트웨어 개발 프로세스의 첫 번째 단계는 무엇을 만들 것인지 결정하는 일
- 무엇을 만들 것인가에 대답은 누가 할 수 있는지? → 개발 프로젝트를 맡은 기업? 또는 개발을 의뢰한 고객사?
  - 개발을 맡은 기업은 시스템을 구현하지만, 고객 업무에 대한 현황과 이상적인 모습, 현재의 문제점을 모두 파악할 수 없음. 따라서 고객이 원하는 시스템의 모습을 개발 업체가 혼자 결정하는 일은 불가능
  - 그러나 개발 기업은 어떻게 시스템을 구성해야 프로그램이 안정되고 효율적으로 작동하는지 알고 있기 때문에 이 부분에서 사용자와 협력하여 무엇을 개발할지 논의할 수 있음
  - 사용자의 요구가 입력되어 도출되는 <요구사항 명세서(Software requirement specification, SRS)>라는 결과물이 나오는 과정은 3개 단계로 구성



요구 분석 절차

- 소프트웨어 개발 과정에서 문제 정의(Problem Definition)는 프로젝트의 출발점이자 가장 중요한 단계 중 하나
  - 해결해야 할 문제를 명확히 이해하고, 프로젝트의 목표와 범위를 정의
  - 문제 정의가 명확하지 않으면 이후의 모든 개발 과정이 잘못된 방향으로 진행될 수 있기 때문에 매우 신중하게 접근해야 함
  - 소프트웨어를 통해 해결하려는 업무가 있음, 거기에는 문제가 존재함, 어떻게 해결하면 좋겠다는 의견이 있을 수 있음
  - 이렇게 소프트웨어 개발에 대한 고객의 제안 요구, 요청을 문서화한 것이 문제 정의 또는 제안 요청서
- 문제 정의에는 중복, 모순, 누락이 있어도 괜찮음. 왜냐하면 뒤따르는 요구 사항 명세서에서 잘 정리되어 시스템으로 구현하기 적합한 형태로 기술되기 때문
- 문제 정의는 본질적으로 고객 자신이 수행해야 하지만 소프트웨어에 대한 지식이 없거나 전체적인 시각이 부족한 고객이 스스로 자신의 문제와 요구를 정의하는 것은 쉽지 않음 → 이러한 이유로 업무 분석가나 소프트웨어 개발 업체의 힘을 빌리는 경우가 많음
- 요구사항 명세는 문제 정의나 제안 요청을 기반으로 고객과 개발 기업이 협력하여 소프트웨어로 구현해야 할 항목, 즉 시스템의 요구사항을 정리해 나가는 단계이며, 정리한 항목을 문서화한 것이 <요구 분석 명세서>
- 공공기관의 경우 개발업체에 용역을 주기 위해 요구 분석 명세서를 작성하는 업무 자체가 공모되기도 함

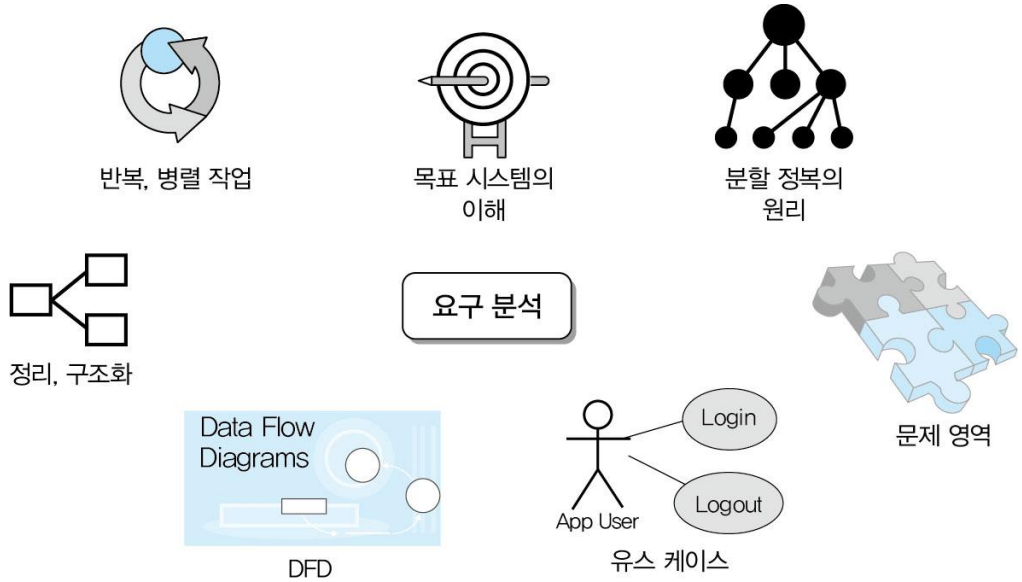
- 요구 분석 명세는 시스템의 기준이 되는 것으로, 중복되거나, 부족하거나 모순이 존재해서는 안됨
- 폭포수 개발 모델의 경우 과/부족이나 모순이 있을 경우 이어지는 설계나 구현 단계에서 수정됨, 그러나 이로 인한 개발 일정 지연 및 비용 증가는 가능한 한 피해야 함
- 결국 요구 분석 명세는 개발 과정 중에 신중하고 완전한 형태로 작성되는 것이 이상적

문제 정의와 요구사항 정의의 차이

	문제 정의(제안 요청서)	요구 사항 정의
작성 주체	고객	개발 회사
주요 내용	배경 개발 목적 SW 개요 시스템 범위	개발 목적 SW 개요 기능적 요구 비기능적 요구 UI, HW/SW 아키텍처, 네트워크, 시스템 인터페이스
완전성	중복, 모순, 누락 존재 가능	중복, 모순, 누락이 없도록 검증되어야 함.
목적	요구 분석, 제안의 기준	개발 전체 활동의 기준
관점	고객, 사용자가 원하는 요구, 해결하기 원하는 문제	SW 시스템으로 구현 가능한 요구, 해결 가능한 문제

프로젝트 성패에 중요한 작업인 요구 분석은 다음과 같은 특징을 가짐

- 분석 작업과 명세 작업은 순차적이 아니라 반복, 병렬 작업
- 분석 작업의 초점은 원하는 소프트웨어 시스템에 대한 이해
- 분할과 정복의 원리, 즉 작은 부분으로 나누고, 각 부분의 관계를 이해하는 전략이 적용
- 상당히 많은 정보를 생성하고 구조화 하는 것이 중요
- DFD, 유스 케이스 다이어그램 등의 기법이 적용됨
- 요구 분석은 설계와 유사한 목적을 둔 작업이지만 문제 영역을 다루며, 설계는 해결 영역을 다룸



요구는 소프트웨어에 대한 고객의 기대사항으로, 기능요구와 비기능 요구로 나눌 수 있음

- 기능 요구: 시스템에 요구되는 처리나 서비스로, 고객의 업무에 필요한 절차를 실현하기 위한 기능
  - 기능 요구의 예: <고객의 이름을 등록>, <고객의 이름을 받아서 주소 찾기>
- 시스템에 요구되는 처리도 아니고 고객의 업무에 필요한 절차도 아니지만 소프트웨어가 갖춰야 할 조건
  - 비기능 요구의 예: <고객 이름으로 주소를 검색하는 과정은 0.5초 이내에 완료되어야 함>, <시스템이 고장난 경우 1시간 이내에 서비스를 재개>와 같은 내용은

기능 요구

- SW에 요구되는 처리 (업무 절차를 구현)
- SW가 제공하는 서비스 기능
- SW가 하여야 할 일
- 사용자의 동작 관점
- 동사로 표현됨.

비기능 요구

- 시스템이 갖추어야 할 조건 (응답 속도, 오류 복구 성능 등)
- SW가 가져야 할 품질 특성
- SW가 하여야 할 일의 특성
- 사용자의 경험 관점
- 형용사로 표현됨.

- 비기능 요구는 시스템의 일부분에 대한 것 보다 전체에 대한 것이 많음
- 또한 기능은 충족하지만 성능, 응답속도, 보안 등의 특성은 고객이 받아들이기 어려운 경우도 많음
- 예를 들어 필요한 기능은 들어 있지만 처리 속도가 늦어서 쓸모가 없다고 판단한다면, 비기능 요구를 만족하지 못하는 소프트웨어로 판별
- 비기능요구를 품질 특성에 따라 포괄적으로 파악하는 것도 좋은 방법이며, 이를 각 품질 특성으로 분류하면 아래와 같음

비기능 요구의 분류

특성	요구에서 정의될 사항
상호운용성, 보안성	필요한 다른 시스템과 연결할 수 있는지, 적절한 보안이 확보되어 있는지에 대한 능력
신뢰성	잠재적 문제를 해결하는 능력, 고장 시 데이터를 복구하는 능력
사용성	사용자가 사용법을 이해하기 쉽게 하거나 표준 사용법에 따르고 있는지를 나타내는 특징
효율성	적절한 응답 시간으로 반응하거나 적절한 메모리를 사용하였는지를 나타내는 특징
유지보수성	결함 진단 및 문제 부분을 식별할 수 있는지, 필요한 수정이 추가되는지를 나타내는 능력
이식성	다른 환경에서도 사용할 수 있거나 다른 소프트웨어와 공존할 수 있는 특징
강인성 (결함 허용성)	장애의 발생을 막을 수 있는지, 장애의 확산을 막을 수 있는 능력
운용성	품질 목표를 충족하고 있는지, 재해 대책은 충분한가를 나타내는 특징

- 이와 같이 성질이 다른 두 가지 요구는 소프트웨어 개발 단계에서 반영되는 방법도 다름
  - 기능 요구는 프로그램 안에서 자료를 처리하는 방법이나 데이터베이스를 구성하는 방법을 결정
  - 비기능 요구는 소프트웨어 아키텍처 설계나 하드웨어와 같은 시스템 구성 요소를 좌우
- 이렇게 성격이 다를 뿐 아니라 영향을 미치는 대상과 설계 방법도 다르기 때문에 두 가지를 구분하고 개별적으로 검토하는 경우가 많음
- 그러나 비기능 요구를 만족시키기 위해 특별한 처리 방식(예: 속도를 향상시킬 수 있는 설계 방식)이 필요하기도 함
- 또한 특수 기능(예: 보안 요구를 충족시키기 위한 암호화 기능)을 도입하는 경우 기능 요구로 분류할 수도 있기 때문에 서로 연계하여 검토할 필요가 있음

인터넷 쇼핑몰의 예

[기능 요구]

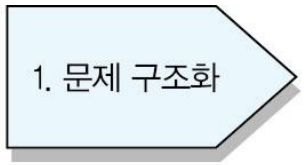
- 고객이 상품을 고르기 위해 열람하고, 구입하려는 물품을 선택하여 주문하는 일련의 구매 절차를 도와주는 기능
- 배송상태와 상품에 대한 만족도 및 후기 올리는 기능 필요
  - 고객의 가입 정보 보관, 유지 기능
  - 상품 정보 보관 유지 기능
  - 일일 매상 보관 유지 기능
  - 고객이 제공한 피드백 저장 기능
  - 상품 주문 및 배송에 대한 세부 정보 보관 유지 기능
  - 고객이 선택한 상품을 임시 저장소에 저장하는 장바구니 기능
  - 단골 고객 확보를 위한 마일리지, 보관, 사용 기능

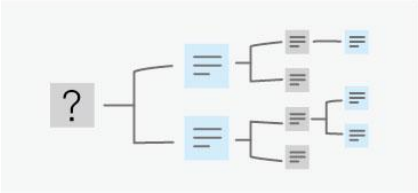
[비기능 요구]

- 시스템의 운용 성능이나 기능을 높이기 위한 제한 조건들 (예: 속도, 보안, 신뢰성)
  - 성능과 확장성: 시간당 5천명의 사용자를 지원하는 방문 페이지는 LTE 연결을 통해 텍스트 및 이미지 렌더링을 포함하여 크롬 브라우저에서 6초 이하의 응답 시간을 제공해야 함
  - 이식성과 호환성: 윈도우 8.x 이상 또는 iOS 3.3 이상의 운영체제에서 호환되어야 함
  - 신뢰성, 가동성, 유지보수성: 웹 대시 보드는 한국 표준시 업무시간(오전 9시-오후 6시)에 매달 98%의 기간 동안 한국 사용자가 사용할 수 있어야 함
  - 보안: 결제 처리 게이트웨이는 PCI DSS를 준수해야 함
  - 사용성: 결제 페이지에서 결제 내부 정보를 제출하는 사용자의 오류율은 10%를 초과할 수 없음

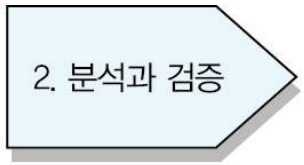
- 요구 분석을 위해 먼저 현황을 분석하고 개발 대상 문제를 이해하는 작업이 필요함. 무엇이 진짜 문제인지 정확히 모르면 정확한 해결책인 소프트웨어 개발을 이끌어 내지 못하기 때문
- 문제라는 것은 2가지 레벨로 구분할 수 있음
  - 눈앞의 문제를 즉석에서 해결할 수 있는 수준
  - 곤란한 문제를 근본적으로 해결하는 수준
- 소프트웨어 개발은 근본적인 문제의 해결이며, 이를 위해서는 심층적으로 분석하는 작업이 필요
- 문제 분석 작업은 현행 시스템을 이해하는 작업이 되기도 하고, 아직 도입되지 않은 경우 수작업의 현황을 파악하는 작업이 될 수도 있음 → 현재 상황을 분석하면 개선할 문제점을 알 수 있고, 해결책을 제시할 수 있음

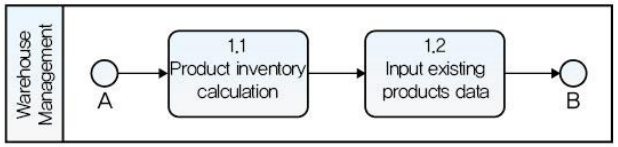
눈앞에 있는 업무의 요소를 분해, 흐름 파악, 구조화, 무엇이 문제이고 그 원인을 파악



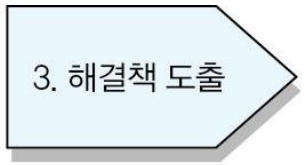


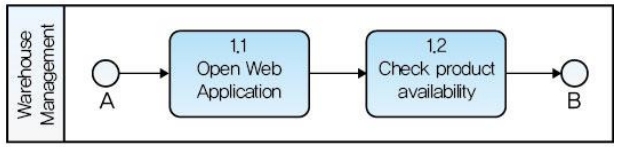
무엇이 중요한 과제인지 우선순위를 정하고 설문이나 인터뷰를 통해 검증





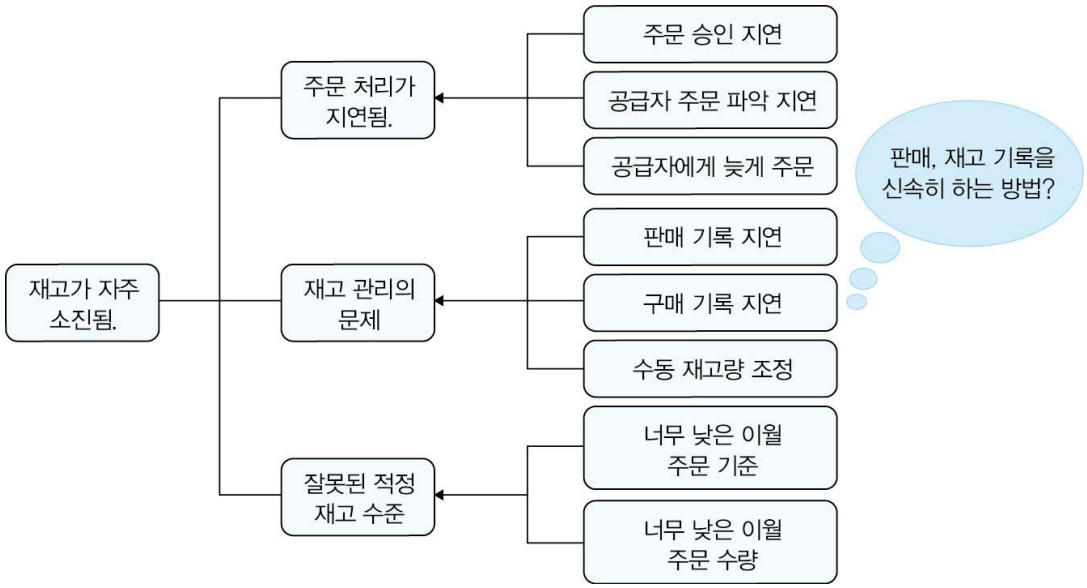
문제를 깊게 파헤치고 원점에서 생각하거나 다른 분야의 문제에 대한 해결책을 참조





문제 분석 과정

- 구조화는 다양한 요소가 혼합하여 존재하는 문제 전체의 모습을 파악하는 것
  - 문제가 어떤 원인에 의해 일어났나?
  - 문제의 핵심이 어디에 있나?
- 문제를 구조화 하는데 사용되는 다이어그램은 로직 트리(이슈 트리)
  - 왼쪽에 문제를 쓰고, 중간에 문제를 원인을 일으키는 원인들을 나열
  - 즉, 문제를 유발하는 여러가지 원인들을 파악하면 증상에 영향을 주는 근본 원인을 찾을 수 있음
- 로직 트리를 이용하여 문제를 구조화할 때 중복과 누락이 없어야 함 (MECE 원리)
- 분할된 문제가 상호 간에 중복되지 않고 전체 문제를 구성하는 요소들이 집합이 되어야 한다는 뜻으로, 이런 방식으로 문제를 구조화 하다 보면 정보를 모으게 되고 현황을 제대로 파악할 수 있음



로직 트리

MECE는 '상호 배타적(Mutually Exclusive)'이면서 '총체적으로 완전(Collectively Exhaustive)'한 분류 방식

1. **상호 배타적**: 각 카테고리는 겹치지 않음. 콜라는 탄산음료에만 속하고, 주스나 물 카테고리에 동시에 속하지 않음
2. **총체적으로 완전**: 모든 종류의 음료수가 이 분류 중 하나에 속함

음료수를 분류하기

좋은 MECE 예시
- 탄산음료 (콜라, 사이다 등)
- 주스 (오렌지주스, 사과주스 등)
- 물 (생수, 탄산수)
- 차/커피 (녹차, 아메리카노 등)

나쁜 MECE 예시
- 차가운 음료
- 뜨거운 음료
- 탄산음료
- 과일음료

→ **문제점: 차가운 음료는 탄산음료와 겹칠 수 있음. 이온음료같은 다른 범주의 음료가 누락되어 있음**

정보시스템 문제 파악하기

좋은 MECE 예시
- 서버 문제 (CPU/메모리 부족, 디스크 공간 부족)
- 네트워크 문제 (대역폭 부족, 연결 불안정)
- 스토리지 문제(입출력 속도 저하, 백업 실패)
- 보안 인프라 문제 (방화벽 오설정, 인증 시스템 장애)

나쁜 MECE 예시
- 심각한 오류
- 사용자 불만
- 성능 문제
- 기술적 문제

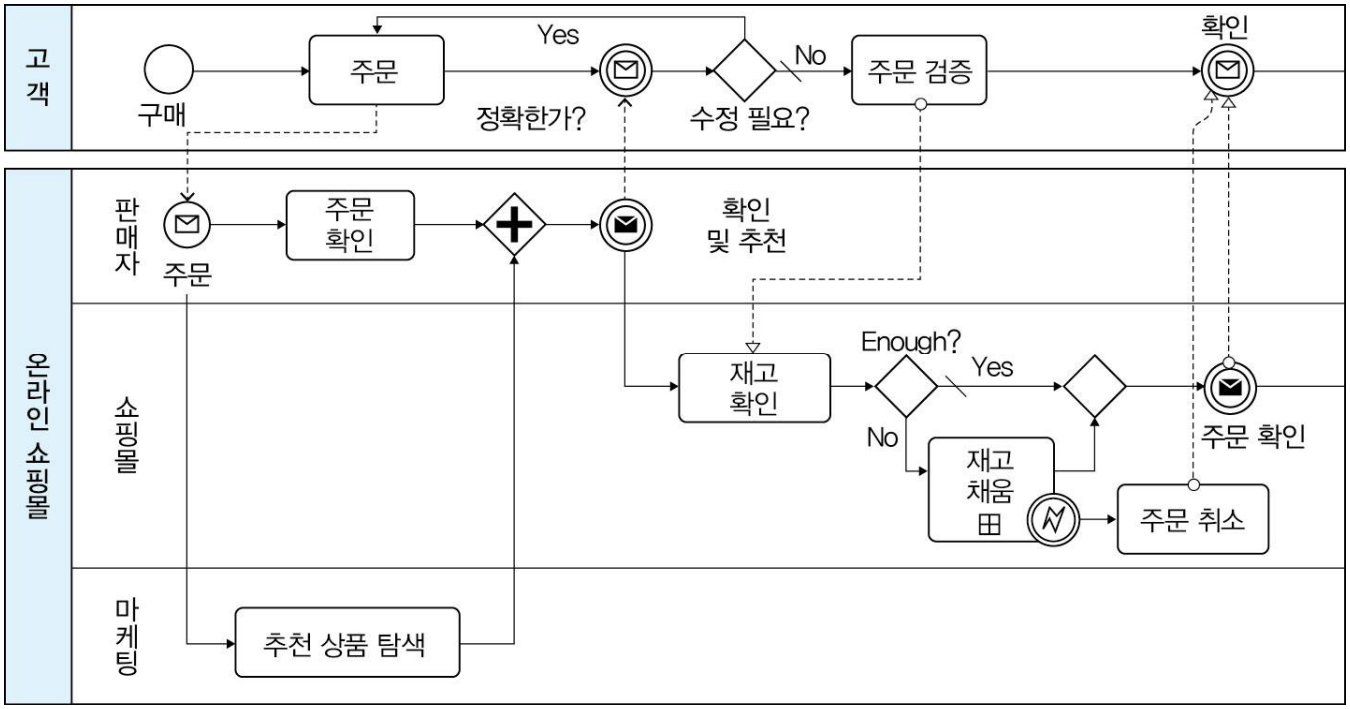
→ **문제점: '심각한 오류'는 성능이나 기술적 문제와 중복될 수 있고, '사용자 불만'은 모든 문제와 연관될 수 있음**

정보 시스템의 소프트웨어 개발은 업무 프로세스와 밀접한 관계

- 업무 프로세스는 업무의 절차, 즉 업무의 흐름을 파악하여 사업에 필요한 요인들을 기능별로 나누어 연속된 흐름으로 정리한 것
  - 일반적인 기업: 개발– 제조 – 마케팅 – 물류 – 도매 영업 - 소매 관리
  - 인터넷 쇼핑: 상품 검색 – 주문 – 재고 확인 – 결제 – 결제 승인 – 주문 확정 – 출고 – 패키징 – 배송 – 배송 완료 – 판매 종료
- 이렇게 업무를 이루는 요소를 분해하고 비교 분석하거나 지연 원인 또는 생략 가능 프로세스를 찾는다면 더 좋은 업무 프로세스를 개발할 수 있음 → 이를 기초로 소프트웨어를 만든다면 업무 효율 향상됨
- 결국 업무 프로세스는 조직 내에서 수행해야 하는 비즈니스의 청사진
  - 일관되고 명확하게 업무 프로세스를 파악하려면 시각적인 업무 흐름 다이어그램을 만들어서 데이터가 한 작업에서 다음 비즈니스 프로세스로 전달될 때 어떤 일이 발생하는지 파악해야 함
  - 소프트웨어로 구현해야 할 업무 프로세스를 실제로 생성하기 전에 시각적으로 표현하면 프로세스가 어떻게 수행되는지 더 잘 파악하고 효율적으로 최적화 할 수 있기 때문

업무 흐름도는 작업과 업무 흐름을 마치 프로그램의 알고리즘으로 나타내는 순서도와 같음

BPMN(Business Process Model and Notation)은 비즈니스 프로세스를 시각적으로 표현하기 위한 표준화된 표기법



업무 흐름도

업무 흐름도는 수영장과 같이 여러 풀로 구성되어 있고, 풀 안은 레인으로 나누어져 있음

고객과 온라인이라는 두 가지 풀의 업무 흐름이 있고, 온라인 쇼핑몰에는 판매자, 마케팅, 쇼핑몰로 구성된 3개 레인으로 구분

업무 흐름도를 그린 후에는 업무 흐름을 개선하기 위한 분석 단계로 이어짐

- 업무 흐름의 효율을 따져보기 위해서는 업무 흐름에 소요되는 시간을 분석하거나 작업 비용을 분석
- 업무 절차가 너무 작은 작업으로 쪼개져 있거나 너무 많은 사람이 투입되어 허비되는 시간이 많은지도 살펴봐야 함
- 여러가지 불필요한 작업을 통합하거나 프로세스를 병렬화 하는 것도 개선안이 될 수 있음

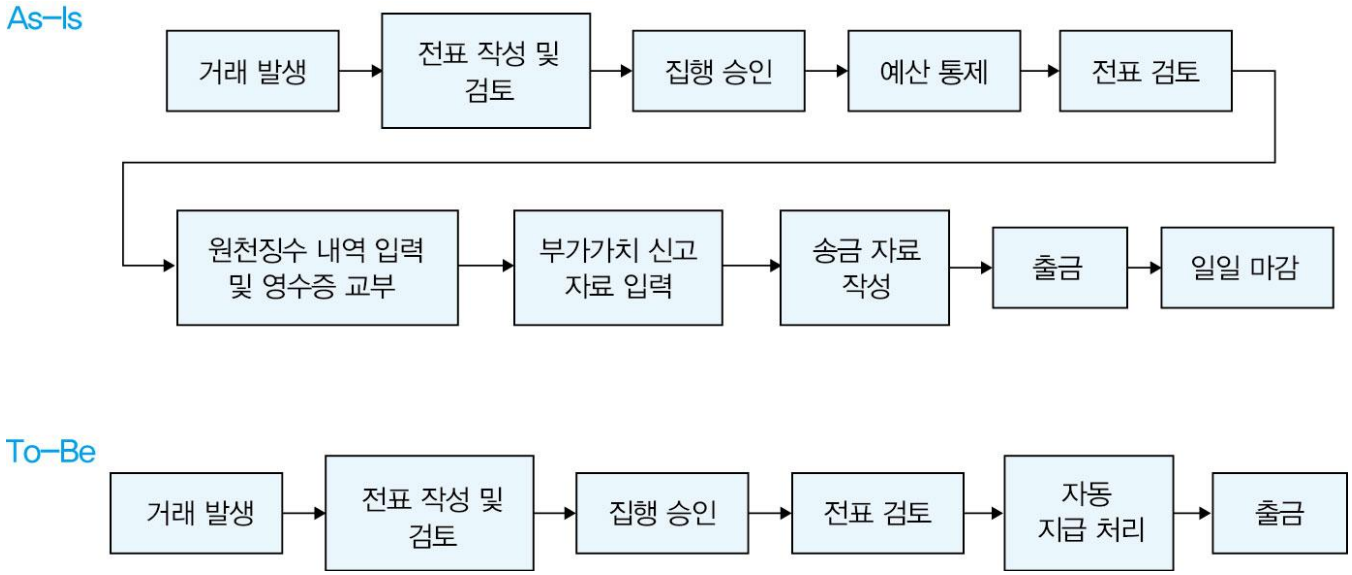
사례	발견된 문제	개선 방안
주문-배송 프로세스에서 병목구간 발견	배송기사 배정 단계에서 평균 30분 지연	• 배송권역 재조정 • AI 기반 자동 배차 시스템 도입 • 성수기 배송기사 인력 풀 확대
구매 승인 프로세스에서 중복 업무 식별	부서장/총무/재무팀에서 동일한 내용 반복 검토	• 승인 단계 간소화 • 금액별 승인 권한 차등화 • 통합 결재 시스템 도입
고객 상담 프로세스에서 너무 긴 대기시간	타부서 협의 필요시 평균 응답 시간 2일 소요	• 부서간 실시간 협업 툴 도입 SLA 수립 및 모니터링 • FAQ 데이터베이스 구축
반품/교환 처리 프로세스	특이케이스 처리시 과도한 수작업 발생	• 예외처리 매뉴얼 구체화 • 권한 위임 범위 확대 • 고객 셀프서비스 기능 강화

분석을 통한 문제 해결의 예

현행 <예산 통제> 업무 프로세스는 전표 입력 시 내역 입력과 영수증 교부, 송금 자료를 입력하게 되어 있음

그러나 구매처가 등록도어 있다면 계좌번호를 참조하여 조회 후 등록된 계좌번호를 가져올 수 있고, 자동 지급 처리 가능하기 때문에 생략 가능한 절차임.

또한 일일 마감 자료는 실시간으로 갱신된 자료를 언제나 볼 수 있도록 기능을 부여하여 프로세스에서 삭제할 수 있음



분석을 통한 문제 해결의 예

To-Be 프로세스는 시뮬레이션 하여 다음 관점의 벤치마킹을 통하여 소프트웨어로 구현할 해결책으로 선택

- 소요 시간 분석: 업무 프로세스를 이루는 각 작업에 소요되는 시간을 자세히 측정하고 총 소요 시간을 계산
- 작업 비용 분석: 업무 프로세스와 관련된 비용을 파악하고, 가장 비용이 많이 드는 업무를 찾아낸 후 이를 개선하는 해결책을 도출

To-Be 프로세스 시뮬레이션

스윔 레인	평균 작업 시간	평균 작업 비용	평균 대기 시간
예산 통제	1.60	2,500	0.61
– 거래 발생	0.32	500	0
– 전표 작성 및 검토	0.70	1,200	0.50
– 집행 승인	0.12	200	0.01
– 자동 지급 처리	0.10	50	0
– 출금	0.06	500	0

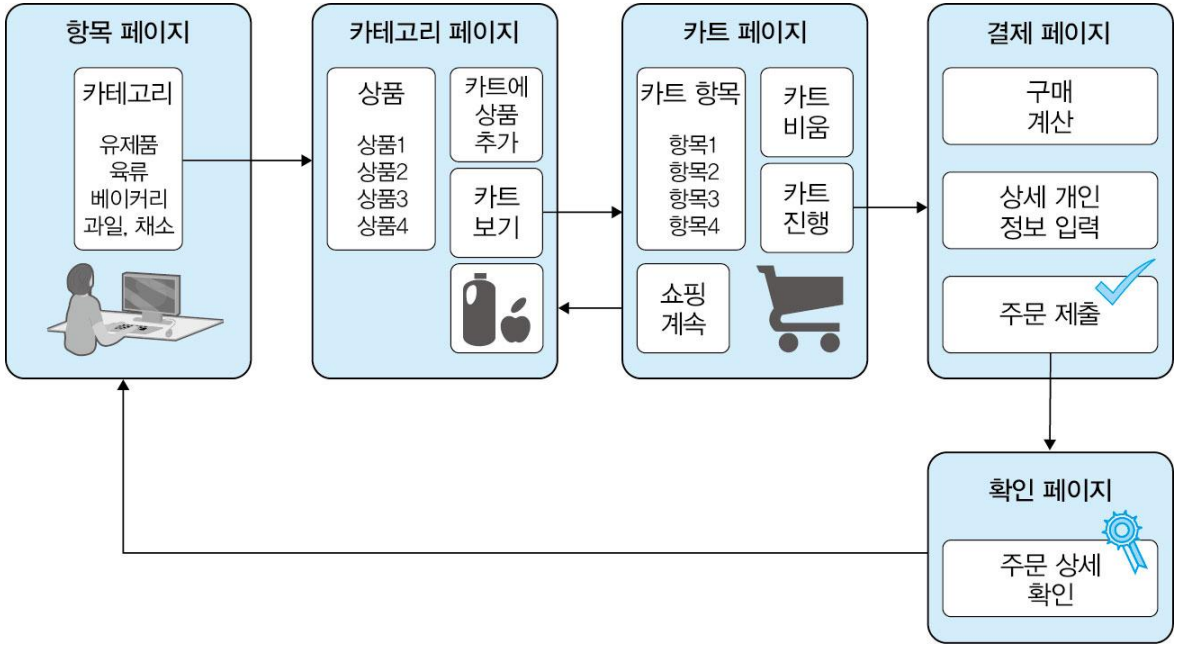
소프트웨어 시스템은 업무 프로세스를 지원하기 위하여 구축하는 것

따라서 소프트웨어 시스템을 이루는 컴포넌트는 To-Be 프로세스의 각 요소들과 관계를 맺고 있으며, 개발 후에 관련된 업무 단계를 지원

예를 들어 웹 기반으로 판매 시스템을 구축한다면, 아래 그림에 있는 각 웹 페이지들이 완성되어 상품을 구매하는 프로세스를 지원하게 됨

결국 요구를 분석하는 일은 문제를 파악하고, 이를 해결하기 위한 업무 프로세스를 설계한 후 소프트웨어가 이를 어떻게 지원할 것인지 궁리하고 상세화하는 작업

업무 프로세스에 대한 소프트웨어 지원



학습정리

- 개발 초기에 명확하지 않은 요구를 요구사항으로 구체화 하는 작업은 시스템 개발 과정에서 필수적인 절차이다.
- 요구 분석은 요구 분석 명세서라는 최종 산출물을 만들어 낸다.
- 요구는 기능 요구와 비 기능 요구가 있으며 이를 빠짐없이 구분하고 정의하는 일은 향후 개발 일정 지연을 최소화 해 준다.
- 요구 분석을 위해서는 현황을 이해하고 개발 대상 문제를 이해하는 작업이 선행되어야 하며, 이 과정에서 문제 구조화와 업무 프로세스 분석을 통해 해결책을 도출할 수 있다.